极客时间

CI/CD 定时触发

从手动到自动

第 2 周 · 第 7 节

讲师：黄佳

极客时间黄佳老师出品

极客时间

本节目标

把 pipeline.py 从"手动跑"变成"每天自动跑"

24/7 无人值守 · 零成本采集 · 按需分析

CI/CD 基本概念

- **CI — Continuous Integration（持续集成）**
- 代码一提交，自动构建、自动测试——有问题立刻发现

- **CD — Continuous Delivery（持续交付）**
- 测试通过后，自动部署到生产环境——人工审批一下就上线

- **对我们的知识库项目来说：**
- CI = 每天自动采集数据 + 自动校验格式
- CD = 采集结果自动提交到仓库 + 自动更新知识库

- **GitHub Actions = 免费的云端自动化平台**
- 不需要自己买服务器，GitHub 提供运行环境
- 公开仓库完全免费，私有仓库每月 2000 分钟免费额度

极客时间黄佳老师出品

GitHub Actions 核心概念

极客时间

Workflow（工作流）
.github/workflows/*.yml

触发器（Triggers）

- schedule→ 定时触发（cron 表达式）
- workflow_dispatch → 手动触发（页面按钮）
- push → 代码推送时触发
- pull_request → PR 创建时触发

Job（任务）
runs-on: ubuntu-latest

Step 1 → Step 2 → Step 3 → ...
(checkout) (setup) (run) (commit)

极客时间黄佳老师出品

Cron 表达式速查

极客时间

表达式	含义	北京时间
0 0 * * *	每天 UTC 00:00	每天 08:00
0 2 * * *	每天 UTC 02:00	每天 10:00
30 1 * * *	每天 UTC 01:30	每天 09:30
0 0 * * 1–5	工作日 UTC 00:00	工作日 08:00
0 0 * * 0	每周日 UTC 00:00	每周日 08:00
0 */6 * * *	每 6 小时	每 6 小时
0 0 1 * *	每月 1 号 UTC 00:00	每月 1 号 08:00

代码展示 — daily-collect.yml

极客时间

.github/workflows/daily-collect.yml | yaml

```yaml
name: Daily Knowledge Collection

on:
  schedule:
    - cron: "0 0 * * *"    # 每天 UTC 0:00（北京 08:00）
  workflow_dispatch:       # 手动触发按钮

jobs:
  collect:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Setup Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.12"
```

.github/workflows/daily-collect.yml | yaml

```yaml
      - name: Install dependencies
        run: pip install -r requirements.txt

      - name: Run collection (Steps 1-2 only)
        run: python pipeline.py --step 1 --step 2

      - name: Commit collected data
        run: |
          git config user.name 'github-actions[bot]'
          git config user.email 'actions@github.com'
          git add knowledge/raw/
          git diff --staged --quiet || git commit -m "daily: auto-collect $(date +%Y-%m-%d)"
          git push
```

极客时间黄佳老师出品

Workflow 执行流程详解

GitHub Actions Runner (ubuntu-latest)

不消耗 LLM tokens!

Step 1：checkout — 拉取仓库最新代码

Step 2: setup-python — 安装 Python 3.12

Step 3: pip install — 安装项目依赖

Step 4: pipeline.py — 执行采集（Steps 1–2，纯 API）

Step 5: git commit + push

- 有新数据 → 自动提交
- 无新数据 → 跳过（不产生空 commit）

极客时间黄佳老师出品

免费采集策略 — 自动化的成本智慧

极客时间

Steps 1–2：自动采集（免费）

GitHub Search API — 免费
Hacker News API — 免费
RSS Feed 解析 — 免费

纯 HTTP 请求，无 LLM 调用
每天自动跑，零 token 成本

→ schedule 定时触发
→ 数据自动积累到仓库

Steps 3–4：AI 分析（按需）

DeepSeek API — ¥0.5–2/次
Qwen API — 按 token 计费
本地模型 — 电费 + 算力

需要 LLM 推理，有成本
手动触发 或 每周一次

→ workflow_dispatch 手动
→ 或 每周日定时摘要

手动触发分析 — workflow_dispatch + 本地运行

极客时间

.github/workflows/daily-collect.yml + 本地终端 | yaml / bash

```yaml
# ── 方式 1：GitHub 页面手动触发 ──────────────────────────────
# Actions → Daily Knowledge Collection → Run workflow
# 点击按钮即可触发完整流程（含 AI 分析）

# 在 yml 中添加输入参数：
on:
  workflow_dispatch:
    inputs:
      run_analysis:
        description: "Run AI analysis (Step 3-4)?"
        type: boolean
        default: false

# Step 中判断是否运行分析：
- name: Run AI analysis
  if: inputs.run_analysis == true
  run: python pipeline.py --step 3 --step 4
  env:
    DEEPSEEK_API_KEY: ${{ secrets.DEEPSEEK_API_KEY }}

# ── 方式 2：本地直接运行 ──────────────────────────────────────
python pipeline.py --step 3 --step 4
```

GitHub Secrets 配置 — 安全管理 API Key

极客时间

- **为什么需要 Secrets?**
- API Key 不能写在代码里——推送到 GitHub 就泄露了
- Secrets 加密存储，只有 Actions 运行时才能读取

- **配置步骤：**
- 1. 打开 GitHub 仓库 → Settings → Secrets and variables → Actions
- 2. 点击 New repository secret
- 3. Name 填 DEEPSEEK_API_KEY，Value 填你的 API Key
- 4. 点击 Add secret

- **在 workflow 中使用：**
- env: DEEPSEEK_API_KEY: ${{ secrets.DEEPSEEK_API_KEY }}

- **安全原则：**
- Secret 一旦保存，无法再查看原始值（只能删除重建）
- Fork 仓库不会继承 Secrets——即使仓库公开，Key 也安全

极客时间黄佳老师出品

opencode run 无人值守 — Headless 模式

极客时间

.github/workflows/ai-analysis.yml | yaml / bash

```
# ── OpenCode 的 headless 模式 ──────────────────────────────────────
# 不启动交互式终端，直接执行指令并返回结果

# 本地使用：
opencode run "读取 knowledge/raw/ 最新数据，执行分析"

# 在 GitHub Actions 中使用：
- name: AI Analysis with OpenCode
  run: |
    opencode run "读取 knowledge/raw/ 中最新的采集数据，
      按照 tech-summary Skill 执行深度分析，
      输出结果到 knowledge/analyzed/"
  env:
    DEEPSEEK_API_KEY: ${{ secrets.DEEPSEEK_API_KEY }}

# ── 适用场景 ──────────────────────────────────────────────────────────
# 1. 需要 AI 决策的自动化任务（不是简单脚本能搞定的）
# 2. 需要调用 Skill 的复杂操作
# 3. 需要 Agent 角色和上下文的任务
```

本地定时任务（备选方案）

极客时间

Linux / macOS: crontab

```
# 编辑 crontab
crontab -e

# 每天 8:00 执行采集
0 8 * * * cd /path/to/project
  && python pipeline.py
  --step 1 --step 2
  >> logs/collect.log 2>&1

# 查看已有定时任务
crontab -l
```

Windows: Task Scheduler

1. 打开「任务计划程序」
2. 创建基本任务
3. 设置触发器：每天 08:00
4. 操作：启动程序
   程序：python
   参数：pipeline.py --step 1
   --step 2
   起始目录：项目路径

```
# 或用 PowerShell:
schtasks /create ...
```

每周知识摘要 — weekly-digest.yml

极客时间

.github/workflows/weekly-digest.yml | yaml

```yaml
name: Weekly Knowledge Digest

on:
  schedule:
    - cron: "0 2 * * 0"   # 每周日 UTC 02:00（北京 10:00）
  workflow_dispatch:

jobs:
  digest:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Setup Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.12"

      - name: Install deps
        run: pip install -r requirements.txt
```

.github/workflows/weekly-digest.yml | yaml

```yaml
      - name: Generate weekly digest
        run: |
          opencode run "读取 knowledge/raw/ 中本周所有采集数据，
            生成本周知识摘要 Markdown 文件，
            输出到 knowledge/digests/week-$(date +%Y-W%V).md"
        env:
          DEEPSEEK_API_KEY: ${{ secrets.DEEPSEEK_API_KEY }}

      - name: Commit digest
        run: |
          git config user.name 'github-actions[bot]'
          git config user.email 'actions@github.com'
          git add knowledge/digests/
          git diff --staged --quiet || git commit -m "weekly: digest $(date +%Y-W%V)"
          git push
```

极客时间黄佳老师出品

监控与告警 — 如何知道自动化是否正常运行?

极客时间

- **1. GitHub Actions 运行状态**
- 仓库 → Actions 标签页 → 查看每次运行的状态（绿色/红色）
- 点击具体运行 → 查看每个 Step 的日志输出

- **2. 邮件通知（默认开启）**
- Workflow 失败时，GitHub 自动发邮件通知仓库 Owner
- Settings → Notifications → Actions 可调整通知策略

- **3. Badge 状态徽标**
- 在 README.md 中添加 Badge，实时显示 Workflow 状态
- ![status](https://github.com/USER/REPO/actions/workflows/daily-collect.yml/badge.svg)
-

- **4. 自定义告警（进阶）**
- 在 Workflow 最后加一个 Step，失败时发 Slack/钉钉通知
- 用上节课学的 MCP 连接外部通知渠道

极客时间

V1 → V2 对比 — 从编码自动化到采集自动化的跨越

极客时间

V1：编码自动化（OpenCode/AI Code）

用 AI 编码搭建 Agent + Skill 骨架
人驱动：每天打开终端运行 pipeline
人检查：手动查看输出结果
人提交：手动 git commit + push

AI 帮你写代码，但执行靠人驱动
忘了就断了，没有连续性

Agent × 3（采集/分析/整理）
每次执行 ~5 分钟人工操作

V2：编码自动化（Hooks/定时作业）

GitHub Actions 定时触发
pipeline.py 自动执行采集
Hooks 自动校验数据格式
自动 commit + push 到仓库

机器永不忘记，24/7 运行
数据连续积累，形成时间线

自动采集 + 手动分析
人工操作 → 0 分钟/天

CI/CD 是贯穿 V2→V3→V4 的自动化底座

极客时间

版本	自动化层次	cron 触发之后发生了什么
V1	[编码自动化]	（无 cron）手动执行 pipeline
V2	[编码自动化] + [CI/CD 定时]	cron → 采集+分析+整理 → JSON 落盘
V3	[编码自动化] + [CI/CD 定时] + [Agent 审核]	cron → 采集+分析+整理 → 多Agent审核 → 合格才落盘
V4	[编码自动化] + [CI/CD 定时] + [Agent 审核] + [Bot 服务]	cron → 采集+分析+整理 → 审核 → 落盘 → 推送Bot/Telegram/飞书

本节作业

极客时间

任务 1：配置 GitHub Actions 定时采集

创建 .github/workflows/daily-collect.yml，配置 schedule 定时触发。推送到 GitHub 后检查 Actions 页面是否正常识别。手动触发一次（workflow_dispatch）验证流程是否跑通。关键检查：knowledge/raw/ 是否有新数据、commit 记录是否正常。

任务 2：本地 crontab 定时运行（Linux/Mac）或 Task Scheduler（Windows）

配置本地定时任务作为备选方案。Linux/Mac 用 crontab -e 添加定时采集任务；Windows 用任务计划程序创建定时任务。注意使用绝对路径、配置日志输出、处理虚拟环境。

任务 3：（可选）添加每周摘要生成

创建 weekly-digest.yml，每周日定时运行。使用 opencode run 的 headless 模式生成周摘要。需要配置 DEEPSEEK_API_KEY（或其他大模型 API Key）到 GitHub Secrets。验证产出的 Markdown 摘要质量——是否有趋势分析、是否有具体数据支撑。

极客时间黄佳老师出品

工程思想总结卡：手动到自动

极客时间

- **核心策略**：免费采集 + 按需分析 = 成本最优的自动化方案

- **自动化 = 把人的工作交给机器**
- 定时采集：schedule cron 触发 → 每天自动跑
- 按需分析：workflow_dispatch 手动触发 → 控制成本
- 无人值守：opencode run headless 模式 → AI 自主执行
- **关键配置**
- .github/workflows/*.yml — GitHub Actions 配置
- GitHub Secrets — 安全管理 API Key
- Badge — 监控 Workflow 运行状态
- **V1 → V2 的跨越**
- 从“依赖人的纪律”到“依赖系统的规则”
- 用确定性替代不确定性——这就是工程化的核心价值

极客时间

本节小结

极客时间

CI/CD = 定时触发 + 自动执行流水线

GitHub Actions cron 表达式控制频率

CI/CD 是底座，贯穿 V2→V3→V4

V2 cron→落盘 / V3 cron→审核→落盘 / V4 cron→审核→推送

极客时间

下一节：第 8 节 · 质量校验与数据清洗